



Object Oriented Software Development

Exceptions & File IO



Agenda & Reading

► Topics:

► Exceptions

- Handling Multiple catch Clauses
- finally
- throw Vs throw

► Simple File IO

► Generics

- Fundamentals of generic types
- Generics & Subtyping
- Type wildcards

► Reading

► The Java Tutorial:

- <https://docs.oracle.com/javase/tutorial/essential/exceptions/>
- <http://docs.oracle.com/javase/tutorial/java/generics/index.html>



1.Introduction

- ▶ Errors occur in software programs. The Java programming language uses exceptions for error handling
- ▶ With exception handling, a program can continue executing (rather than terminating) after dealing with a problem.
 - ▶ i.e. improve your program's readability, reliability and maintainability.
- ▶ Exception examples:
 - ▶ `ArithmeticException`: attempt to divide by ZERO
 - ▶ `ArrayIndexOutOfBoundsException`: use of an invalid array index
 - ▶ `NullPointerException`: an attempt to use an object reference that hasn't yet been assigned to an object
 - ▶ `FileNotFoundException`: couldn't find the specific file to read
- ▶ The Java run-time system will attempt to handle the exception (default exception handler), usually by displaying an error message and terminating the program. For example:

```
> java Example1  
Exception in thread "main" java.lang.ArithmeticException: / by zero  
at Example1.main(Example1.java:4)
```



1.Exceptions

What is an Exception?

- ▶ An exception is an event that occurs during the execution of a program that disrupts the normal flow of instructions during the execution of a program.
- ▶ When an error occurs within a method, the method creates an exception object and hands it off to the runtime system (we called “Exceptions are thrown”)
 - ▶ The exception object contains information about the **error**, including its **type** and the **state** of the program when the error occurred.
 - ▶ Information includes:
 - ▶ The **name** of the exception in a descriptive **message** that indicates the problem that occurred
 - ▶ The **method-call stack** (i.e., the call chain) at the time it occurred.
Represents the path of execution that led to the exception method by method.
 - ▶ This information helps you debug the program.

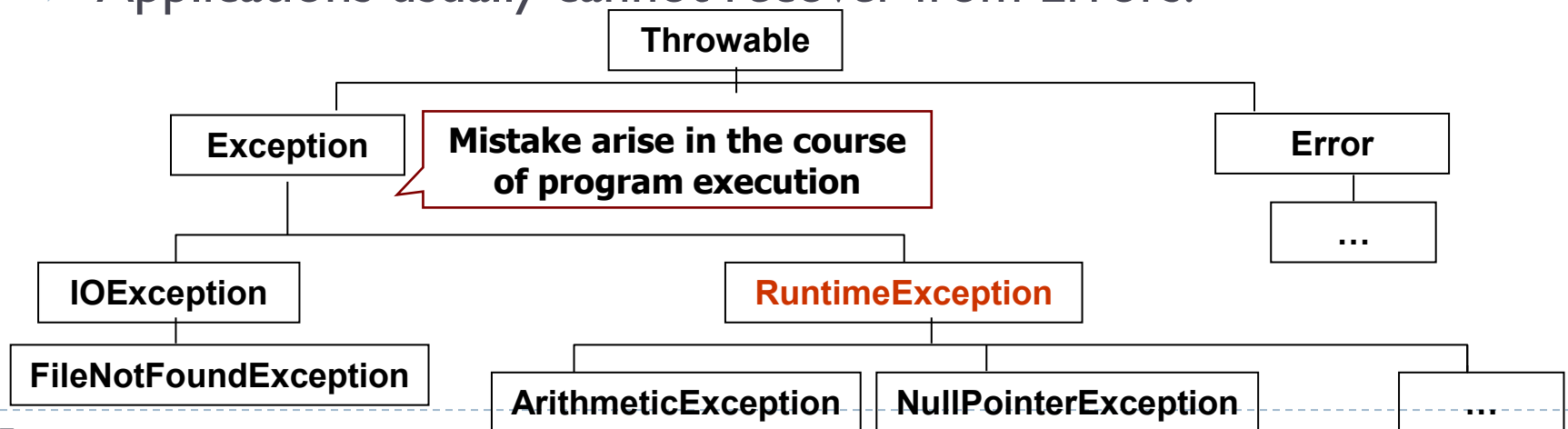


1.Exceptions

Java Exception Hierarchy

- ▶ Class Exception and its subclasses represent exceptional situations that can occur in a Java program
 - ▶ These can be caught and handled by the application.
- ▶ Class Error and its subclasses represent abnormal situations that happen in the JVM.
 - ▶ Errors happen infrequently.
 - ▶ These should not be caught by applications.
 - ▶ Applications usually cannot recover from Errors.

Serious and unrecoverable exceptions, such as running out of memory or Unable to load a needed class





1.Exceptions

Handling exceptions

- ▶ When errors occur, your program ought to
 - ▶ Either return to a safe state and enable the user to execute other commands, or
 - ▶ Allow user to save all his work and terminate the program gracefully.
- ▶ How?
 - ▶ After a method throws an exception, the runtime system attempts to find something to handle it.
 - ▶ If the runtime system could **not** find an **appropriate exception handler**, the runtime system (and, consequently, the program) terminates.
 - ▶ When an appropriate handler is **found**, the run-time system passes the exception to the handler.
 - ▶ An exception handler is considered appropriate if the **type** of the exception object thrown matches the **type** that can be handled by the handler. The exception handler chosen is said to catch the exception



1. Exceptions

No Exception Handler

Examples: Example1.java
Example2.java

- ▶ The exception is caught by the default Exception Handler
 - ▶ The java run-time system
 - ▶ displays a string describing the exception and prints a stack trace from the point at which the exception occurred, and
 - ▶ terminates the program.

▶ Example 1

```
int value = 0;  
int answer = 100 / value;  
System.out.println("This will not be printed");
```

```
> java Example1  
Exception in thread "main" java.lang.ArithmeticException: / by zero  
at Example1.main(Example1.java:4)
```

▶ Example 2

```
int[] array = new int[1];  
array[0] = 1;  
array[1] = 2;           // ERROR  
System.out.println("This will not be printed");
```

```
> java Example2  
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: 1  
at Example2.main(Example2.java:6)
```



1.Exceptions

Basic try-catch block

▶ try block

- ▶ Place the code that might cause the exception in a try block
- ▶ When error happens, Java **ignore** the rest of the code in the try block and **jumps** to the catch block

▶ catch block

- ▶ Specify the exception that you wish to catch, or catch the general exception (e.g. Exception e) Since all exceptions are subclasses of the Exception class, you can catch all exceptions using this way.
- ▶ **Execute** the code if the exception is thrown
- ▶ **Skip** the code if no exception

▶ Statements after the catch block

- ▶ Execute if either the exception is not thrown or if it is thrown

```
try {  
    ...  
} catch (Exception e) {  
    System.out.println("Error");  
}  
System.out.println("continue to run...");
```




1. Exceptions try-catch

```
> java Example3
```

```
_____ <- 2
```

```
_____ <- 3
```

```
try {  
    value = 0;  
    int answer = 100 / value;  
    System.out.println("This will not be printed"); //1  
  
} catch (Exception e) {  
    System.out.println("Error"); //2  
    value = -1;  
}  
System.out.println("Program can continue to run..."); //3
```

Example: Example3.java

► Note:

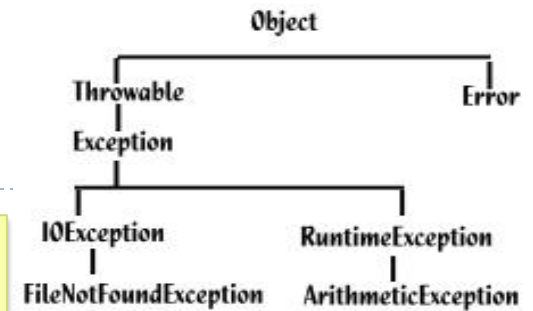
1. “This will not be printed” – this statement is never executed //1 (in case of error, control jumps out from the `try` block to the catch block)
2. The `catch` block catches all types of exception. It prints an error message. //2
3. By catching the exception, it allows the program to continue executing.



1.Exceptions

Example 4

```
try {  
    int[] array = new int[1];  
    array[0] = 1;  
    array[1] = 2;           // ERROR  
    System.out.println("This will not be printed");  
} catch (ArrayIndexOutOfBoundsException e) {  
    System.out.println("ERROR!");  
}  
System.out.println("Finish! ");
```



Example: Example4.java

```
> java Example4
```

► Note:

- “This will not be printed” – this statement is never executed.
- The `catch` block catches the `ArrayIndexOutOfBoundsException`.
 - The argument type of the `catch` block, `ExceptionType`, declares the type of exception that the handler can handle and
 - It must be the name of a class that inherits from the `Throwable` class.
 - The system considers it a match if the exception object can legally be assigned to the exception handler's argument.
- By catching the exception, it allows the program to continue executing.



2. Multiple catch clauses

Handling Multiple catch clauses

- ▶ You can put **two or more** catch clauses, each catch block is an exception handler and handles the **type of exception** indicated by its argument in a program.
- ▶ No code can be between the end of the try block and the beginning of the first catch block.
- ▶ The runtime system invokes the exception handler when the handler is the **first one that matches** the type of the exception thrown.
- ▶ It executes the statement **inside** the matched catch block, the other catch blocks are **bypassed** and **continues** after the try-catch block.
- ▶ Consider the following cases:
 - ▶ No Exception
 - ▶ Exception (e.g. NumberFormatException)
 - ▶ Uncaught exception - there are no matching catch blocks.
 - ▶ General Exception (Superclass)
 - ▶ Order of Catch Clauses

```
try { ...  
} catch (ExceptionType name) {...  
} catch (ExceptionType name) {...  
} ...
```



2. Multiple catch clauses

Case 1: No Exception

Example: Example5_1.java

```
try {  
    String c = "10";  
    int r1 = Integer.parseInt(c);  
  
} catch (NumberFormatException e) {  
    System.out.println("Incorrect Format!");  
  
} catch (Exception e) {  
    System.out.println("General Exception Error!");  
}  
System.out.println("Finished!");
```

>java Example5_1

"12" → 12 ✓

"a" → ✗

► Note:

- No exception occurs in the above program
- The runtime system executes all statements within the `try` block
- It skips all statements in all `catch` blocks
- It continues to run with the first statement after the catch blocks until the end



2. Multiple catch clauses

Case 2: Exception

>java Example5_2



```
try {  
    String c = "abacde";  
    int r1 = Integer.parseInt(c);  
  
} catch (NumberFormatException e) {  
    System.out.println("Incorrect Format!");  
  
} catch (Exception e) {  
    System.out.println("General Exception Error!");  
}  
System.out.println("Finished!");
```

► Note:

- If an exception occurs in a try block, the try block **terminates** immediately.
- The java run-time system scan sequentially for an exception type **matching** the exception that was thrown
- By catching the `NumberFormatException`, it
 - executes the code in the matching catch block,
 - skips any remaining catch blocks,
 - continues after the try-catch block.



2. Multiple catch clauses

Case 3: Uncaught exception

Example: Example5_3.java



```
try {  
    String c = "abacde";  
    int r1 = Integer.parseInt(c);  
  
} catch (ArrayIndexOutOfBoundsException e) {  
    System.out.println("Error!");  
  
} catch (ArithmeticException e) {  
    System.out.println("Calculation Error!");  
}  
System.out.println("Finished!");
```

**throws
NumberFormatException**

```
>java Example5_3  
Exception in thread "main" java.lang.NumberFormatException: For input string:  
"abcde"  
...
```

► Note:

- If **no matching catch block** is found, the Java run-time system will attempt to handle the exception, by terminating the program.



2. Multiple catch clauses

Case 4: General Exception

Example: Example5_4.java

```
try {  
    String c = "abacde";  
    int r1 = Integer.parseInt(c);  
} catch (ArithmeticException e) {  
    System.out.println("Calculation Error !");  
}  
} catch (Exception e) {  
    System.out.println("General Exception Error!");  
}  
System.out.println("Finished!");
```

**throws
NumberFormatException**

matched

```
java Example5_4
```

► Note:

- Exceptions are arranged in an inheritance hierarchy.
- A `catch` specifying an Exception near the top of the hierarchy (a very general Exception) are match any Exception in the subtree.



2. Multiple catch clauses

Order of catch clauses

Example: Example5_5.java

```
try {
    String c = "abacde";
    int r1 = Integer.parseInt(c);

} catch (Exception e) {
    System.out.println("General Exception Error!");
} catch (NumberFormatException e) {
    System.out.println("Incorrect Format!");
}
System.out.println("Finished!");
```

Example5_5.java:10: exception java.lang.NumberFormatException has already been caught

```
    } catch (NumberFormatException e) {
        ^
```

- ▶ Specific exceptions are derived from more general types
 - ▶ Both the specific and general types from which they are derived will handle exceptions of the more specific type
 - ▶ So Exception subclass (specific type of exception) must come before any of their superclass (general Exception)
- ▶ If not, the program can't be compiled



2. Multiple catch clauses

Exercise 1

- ▶ What is the output of the following code fragment?

```
int num = 0;
try {
    num = Integer.parseInt( "0" );
    num = 100 / num;
    System.out.print( "A" );

} catch( NumberFormatException e ) {
    System.out.print( "User error" );
    num = -1;

} catch( ArithmeticException e ) {
    System.out.print( "Division by 0" );
    num = -1;

}
System.out.print( "Number: " + num );
```



3.Finally

- ▶ Code within a `finally` block is **guaranteed to be executed** if any part of the associated `try` block is executed regardless of an exception being thrown or not.
- ▶ It allows for cleanup of actions that occurred in the `try` block but may remain undone if an exception is caught
- ▶ Guaranteed to be executed in the event that code in the associated `try` executes a `return`, `continue`, or `break`
- ▶ Consider the following cases:
 - ▶ No Exception
 - ▶ Exception
 - ▶ Exception is Not Matched
 - ▶ Return



3.Finally No Exception

```
>java Example8_1
```

Example: Example8_1.java

```
try {  
    String a = "100";  
    int r2 = Integer.parseInt(a) / Integer.parseInt(a);  
} catch (ArithmeticException e) {  
    System.out.println("Calculation Error");  
} catch (Exception e) {  
    System.out.println("General Exception");  
} finally {  
    System.out.println("Finally");  
}  
System.out.println("Finished");
```

► Note:

- The `try` block runs to the end and no exception is thrown.
- The `finally` block runs after the `try` block/.



3.Finally Exception

>java Example8_2

Example: Example8_2.java



```
try {
String a = "0";
int r2=Integer.parseInt(a) / Integer.parseInt(a);
} catch (ArithmeticException e) {
System.out.println("Calculation Error");
} catch (Exception e) {
System.out.println("General Exception");
} finally {
System.out.println("Finally");
}
System.out.println("Finished");
```

► Note:

- An exception is thrown in the `try` block and caught in the matching `catch` block
- The `finally` block runs after the `catch` block



3.Finally Not Matched

Example: Example8_3.java

```
>java Example8_3
```

```
Exception in thread ...
```

```
try {  
    String a = "abcde";  
    int r2=Integer.parseInt(a) / Integer.parseInt(a);  
} catch (ArrayIndexOutOfBoundsException e) {  
    System.out.println("Error");  
} catch (ArithmeticException e) {  
    System.out.println("Calculation Error!");  
} finally {  
    System.out.println("Finally block");  
}  
System.out.println("Finished");
```

► Note:

- An exception is thrown in the `try` block and there is no matching `catch` block
- The `finally` block is executed before the method ends
- Code that is after the `catch` block but not in a `finally` block would not be executed in this situation (“Finished” is not shown)
- The exception is caught by the default exception handler and program terminates



3.Finally Return

```
>java Example8_4
```

Example: Example8_4.java



```
try {  
    String a = "0";  
    int r2 = Integer.parseInt(a) / Integer.parseInt(a);  
} catch (ArithmeticException e) {  
    System.out.println("Calculation Error");  
    return;  
} catch (Exception e) {  
    System.out.println("General Exception");  
} finally {  
    System.out.println("Finally block");  
}  
System.out.println("Finished");
```

► Note:

- Code within a `finally` block is guaranteed to be executed, even in `return`
- The `finally` block is executed before the method ends
- Code that is after the `catch` block but not in a `finally` block would not be executed in this situation (“Finished” is not shown)

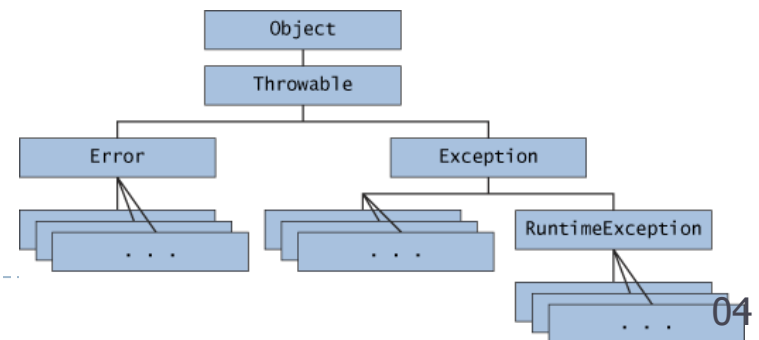


Explicitly Throw Exceptions

- ▶ We can throw an exception explicitly using the throw statement

```
if (condition)
    throw new Exception("Exception occurs.");
```

- ▶ The **throw** statement requires a single argument: a **throwable object**. Throwable objects are instances of any subclass of the **Throwable** class.
- ▶ **Result:**
 - ▶ The program stops immediately after the **throw** statement; and any subsequent statements are not executed.
- ▶ It is normally used in testing and debugging purpose





4.throw & throws

Example

Example: Example9.java

```
>java Example9
```

```
java.lang.Exception: Throw an error
```

```
try {
    System.out.println("main+");
    int i = 0;
    if (i <= 0)
        throw new Exception("Throw an error"); → raise in pytho
    System.out.println("main-");
} catch (Exception e) {
    System.out.println(e);
} finally {
    System.out.println("Finally block!");
}
System.out.println("Finished!");
```

► Note:

- The program throw an exception explicitly when i is equals to 0.
- The code after the exception is not executed. ("main-" is not shown)
- The exception is caught by the `catch` block and the program continues to run until the end.



4.throw & throws

Exercise 2

- What is the output of the following code fragment?

```
int num = 8;
if (num % 2 == 0) {
    throw new ArithmeticException("Incorrect input ");
}
try {
    System.out.print("A ");
} catch (ArithmeticException e) {
    System.out.print(e);
}
System.out.print("B ");
```



4.throw & throws throws

- ▶ A `throws` clause lists the exceptions that can be thrown by a method
- ▶ Sometimes, it's better to not catch the exception and to allow a method further up to handle it.
 - ▶ The method must specify the exceptions that it can throw instead of catching them
 - ▶ Add a `throws` clause to the method declaration. The `throws` clause comprises the `throws` keyword followed by a comma-separated list of all the exceptions thrown by that method.

```
public void method1() throws IOException {  
    ...  
}
```

```
public void method2() throws IOException,  
                           ArrayIndexOutOfBoundsException{  
    ...  
}
```



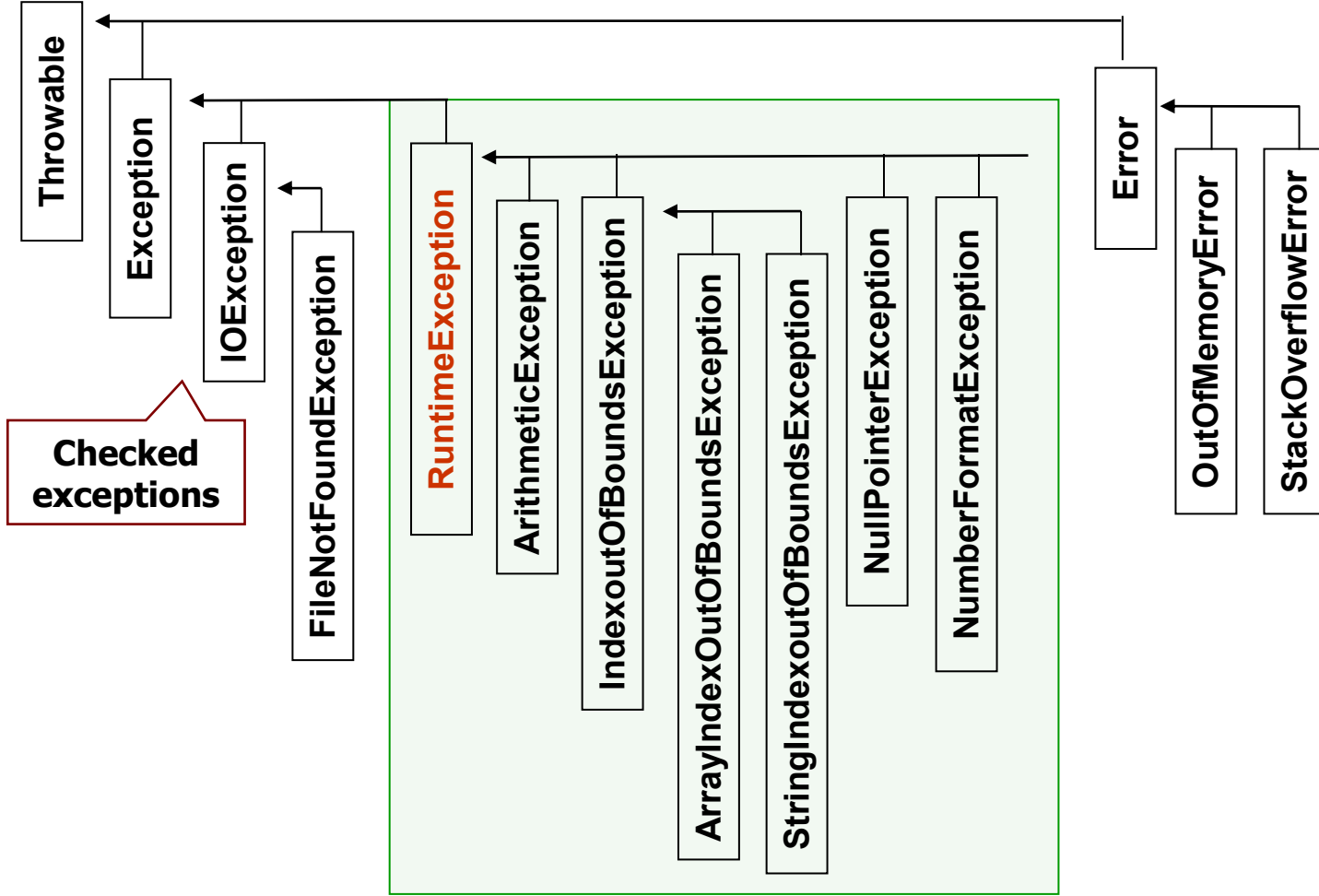
4.throw & throws

Checked Exceptions

- ▶ Compiler enforces a catch-or-declare requirement for checked exceptions.
 - ▶ If a method throws a checked exception, it expects the `catch` block to be in that method unless it is **deferred** by a `throws`-clause.
 - ▶ If the catch-or-declare requirement is not satisfied, the compiler will issue an error message.
- ▶ If the calling method also defers with a `throw`-clause, its calling program is expected to have the `catch` block, etc., up the line all the way to the main method, until a `catch` block is found.
 - ▶ Runtime exceptions:
 - ▶ Occur within the Java runtime system: arithmetic exceptions, pointer exceptions and indexing exceptions
 - ▶ Non-runtime exceptions
 - ▶ Occur in code outside of the Java runtime system. (e.g. exceptions that occur during I/O)
 - ▶ The compiler ensures that non-runtime exceptions are caught or specified; thus, they are also called checked exceptions.



4.throw & throws Exceptions





4.throw & throws try & catch Vs throws

Examples: Example1_1.java
Example1_2.java

```
public static void main(String[] args) {  
    try {  
        throw new Exception("Throw an error");  
    } catch (Exception e) {  
        System.out.println(e);  
    }  
}
```

```
public static void main(String[] args) throws Exception {  
    throw new Exception("Throw an error");  
}
```

► Which one should I use?

► Use try and catch

- If it's clear what action should be taken and if the local block can perform that action.
 - The java runtime system catches the exception and continues to run until the end even it reaches an error.

► Use throws

- Where it's not clear what action should be taken
 - The program terminates when an exception is thrown inside the main method.



Exercise 4

- ▶ Consider the following:

```
public static int getSum(String[] values) {  
    int result = 0;  
    for (String s: values)  
        result += Integer.parseInt(s);  
    return result;  
}
```

- ▶ Rewrite the above class method using a try-catch clause to handle any exceptions that may occur.
 - ▶ encounters an invalid value, the method should **abort** the execution and returns the sum up to but not including the invalid value

"1", "2", "A", "B", "4"

Error: Invalid value!
The result is: 3

"9", "A", "12", "21", "A", "B", "32"

Error: Invalid value!
The result is: 9



Exercise 5

- ▶ Consider the following:

```
public static int getSum(String[] values) {  
    int result = 0;  
    for (String s: values)  
        result += Integer.parseInt(s);  
    return result;  
}
```

- ▶ Rewrite the above class method using a try-catch clause to handle any exceptions that may occur.
 - ▶ ignore any invalid values and continue to calculate the sum

"1", "2", "A", "B", "4"

The result is: 7

"9", "A", "12", "21", "A", "B", "32"

The result is: 74



5.File IO Streams

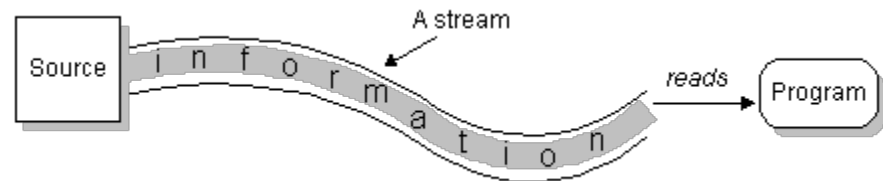
- ▶ **Stream:**

- ▶ An object that either delivers data to its destination (screen, file, etc.) or that takes data from a source (keyboard, file, etc.)
- ▶ It acts as a buffer between the data source and destination

- ▶ **Input stream**

- ▶ A stream that provides input to a program
- ▶ Example:

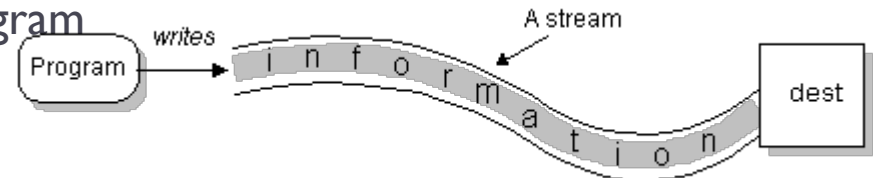
`System.in`



- ▶ **Output stream**

- ▶ A stream that accepts output from a program
- ▶ Example:

`System.out`



- ▶ Algorithms for sequentially reading and writing data are basically the same:
- ▶ We interact with text files (i.e. characters stream)

Reading:

```
Open a stream
While more information
  Read information
Close the stream
```

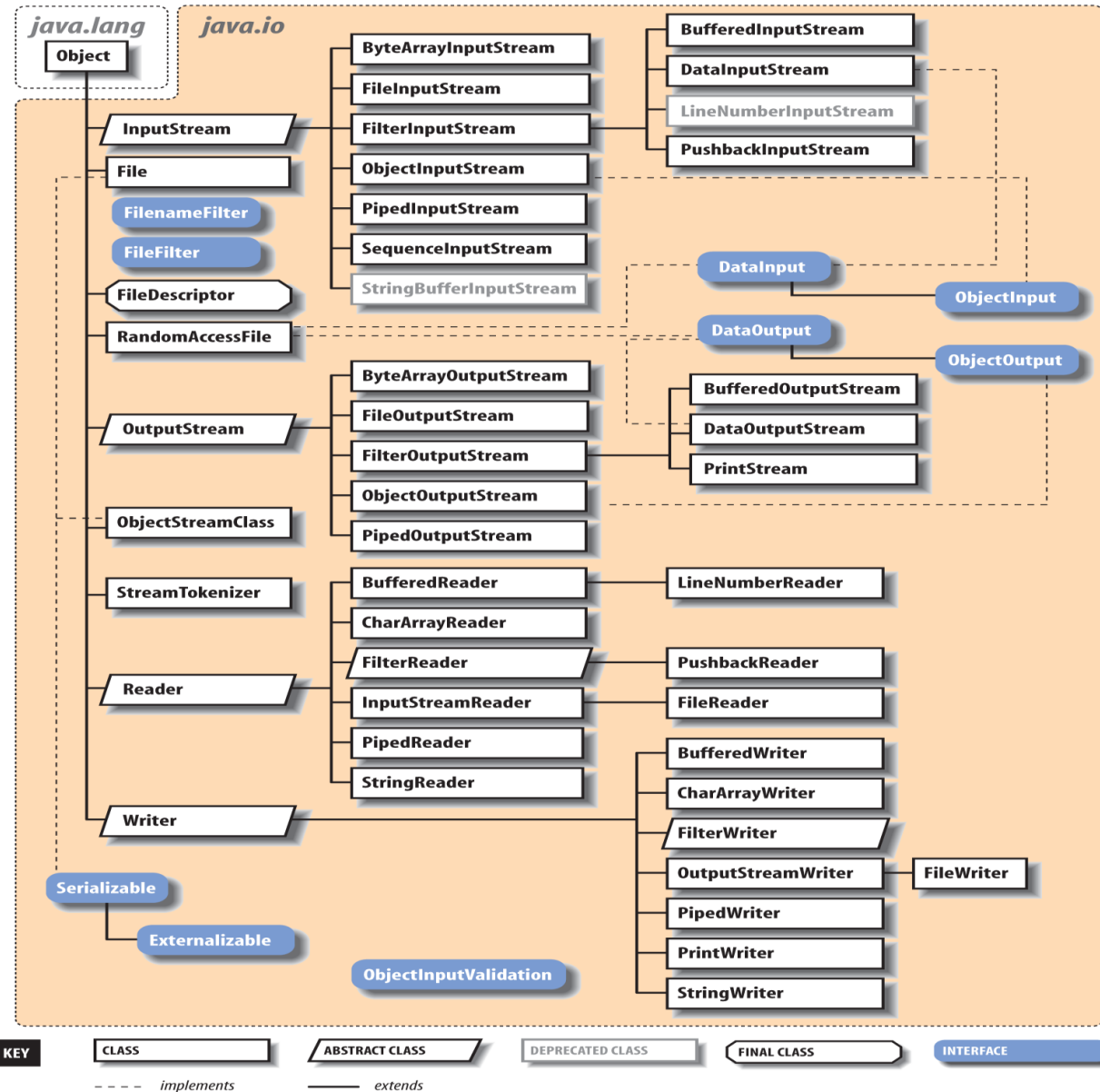
Writing:

```
Open a stream
While more information
  Write information
Close the stream
```



5. File IO

- ▶ All Java I/O classes are defined in the `java.io` package
- ▶ To use these classes, a program needs to import the `java.io` package





5. File IO

Exception Handling

- ▶ When performing file I/O there are many situations in which an exception may be thrown
 - ▶ `FileNotFoundException`
 - ▶ Many of these exception classes are subclasses of the class `IOException`
 - ▶ The class `IOException` is the root class for a variety of exception classes having to do with input and/or output
 - ▶ These exception classes are all checked exceptions. (require you to handle the exception in some way)
 - You can either handle the exception with a catch-block, or
 - defer it with a throws-clause.

```
try {  
    // open a file  
    // Read/write information  
} catch (IOException e) {  
    System.out.println("ERROR");  
} finally {  
    //close the file  
}
```

```
public static ... throws IOException {  
    //Open a file  
    //Read/write information  
    //Close the file  
}
```



5. File IO

Reading from text files

- ▶ Since opening a file can result in an exception, it should be placed inside a try block
- ▶ Create a File instance to represent a text file and passed the object to java.util.Scanner for scanning.
 - ▶ Declare the scanner object outside the block so that it can be closed in the finally block
 - ▶ Use hasNextLine() and nextLine() to read the file line by line
 - ▶ Close the scanner object in the finally block

```
Scanner input = null;
try {
    input = new Scanner(new File("life.txt"));
    while (input.hasNextLine()) {
        System.out.println(input.nextLine());
    }
} catch (Exception e) {
    e.printStackTrace();
} finally {
    input.close();
}
```



5. File IO Scanner

- ▶ The scanner breaks input into individual tokens associated with bits of data
 - ▶ By default, a scanner uses white space to separate tokens
 - ▶ blanks, tabs, and line terminators
 - ▶ Methods:
 - ▶ boolean hasNext()
 - ▶ String next()
 - ▶ boolean hasNextInt()
 - ▶ int nextInt()
 - ▶ Scanner useDelimiter(Pattern pattern)
 - Sets this scanner's delimiting pattern to the specified pattern

```
s.useDelimiter(",");
```

```
input = new Scanner(new File("names.txt"));  
input.useDelimiter(",");  
while (input.hasNext()) {  
    System.out.println(input.next());  
}
```

Alan, Michael, Helen, Bob, Peter, John, Alan, Anna

content of a text file

read a list of names from a text file.
Names are separated with a “,”



Exercise 6

► Consider the following

```
public static String getString(String filename) {  
    Scanner fileInput = null;  
    String line = null;  
    StringBuffer sb = new StringBuffer();  
    try {  
        ...  
    }
```

► Reads the contents of the text file line by line.

► Returns a string

- The string contains the first letter of each line
 - Use try-catch-finally and close the input file
 - Use hasNextLine() and nextLine() to read each line
 - Get the first letter

```
System.out.println(getString("words.txt"));
```

```
Answer reply respond retort acknowledge  
Bright shining shiny gleaming  
Cool chilly cold frosty wintry icy
```

ABC



Exercise 7

► Consider the following

```
public static void printNames(String filename) {  
    Scanner fileInput = null;  
    String line = null;  
    StringBuffer sb = new StringBuffer();  
    try {  
        ...  
    }
```

► Reads the contents of the text file

► Prints a list of names

- Create a scanner object
- Use try-catch-finally and close the input file
- Use useDelimiter(",")
- Use hasNext() and next() to read each name

```
printNames("names.txt");
```

```
Abby,Bella,Charlotte
```

```
Abby  
Bella  
Charlotte
```